

Tensor Train (TT) Decomposition and PDE Solvers

Pavel Altmann, Jakub Kopčil, Eva Trejbalová (WREN)

June 2, 2026

This document collects definitions and core algorithms for the tensor train (TT) format, basic operations in TT, and how TT representations can be used to solve higher-dimensional (4–10 dimensions) partial differential equations (PDEs) in Python using NumPy.

Contents

1	Motivation	3
2	Tensor Definition	3
3	Tensor Train Decomposition	4
3.1	Definition	4
3.2	TT-rank and related concepts	6
4	Basic Operations in TT Format	6
4.1	Computing TTD	6
4.1.1	From Unfoldings to TT-SVD	6
4.1.2	Algorithm for computing a TT decomposition (TT-SVD)	6
4.2	Rounding	8
4.2.1	Algorithm for rounding (rank truncation / recompression)	8
4.3	Addition	10
4.4	Inner products	11
4.5	Matrix-vector multiplication	12
4.6	Flipping an axis	13
4.7	Element access	14
4.8	Mode permutation (transpose)	14
4.9	Tensor contraction (tensordot)	16
5	TT-Matrix Format	17
5.1	TT-Matrix representation	17
5.2	Differential operators in TT format	17
5.2.1	Gradient operator	17
5.2.2	Laplace operator	18
5.3	TT-matrix applied to TT tensor	19
6	PDEs and Using TT format to solve them	20
6.1	Definition of PDE	20
6.2	Discretization and the Curse of Dimensionality	21
6.3	PDEs with Constant Coefficients	22
6.4	PDEs with Variable Coefficients	23
6.5	Structural Lattice of PDEs	23
7	Implicit Methods for Solving PDE	24
7.1	Implicit Method	24
7.2	Method A	25
8	Explicit Methods for Solving PDE	25
8.1	Explicit Method	25
8.2	Method A	26
9	Conclusion and Further Reading	26

1 Motivation

The goal of the Tensor Train (TT) decomposition is to represent such tensors using a number of parameters that grows only linearly in d , provided certain low-rank structure exists.

High-dimensional problems naturally lead to tensors, i.e., multidimensional arrays of numbers.

If a function

$$f(x_1, \dots, x_d)$$

is sampled on a tensor grid with n_k points in each dimension, the resulting data structure is a tensor

$$A \in \mathbb{R}^{n_1 \times \dots \times n_d}.$$

Storing such a full tensor requires

$$n_1 n_2 \dots n_d$$

numbers, which grows exponentially in d . This phenomenon is known as the **curse of dimensionality** [Ose09, Ose11]. Low-rank tensor formats aim to overcome this exponential growth.

2 Tensor Definition

Definition 2.1 (Tensor). A d -dimensional tensor A is a multidimensional array

$$A \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d},$$

whose entries are indexed as

$$A(i_1, \dots, i_d), \quad 1 \leq i_k \leq n_k.$$

Explanation (informal). A tensor is simply a generalization of vectors and matrices to dimensions $d > 2$. For example, a video can be thought of as a tensor with 4 dimensions: width $\times$ height $\times$ color channels $\times$ time. The problem is that storing all entries quickly becomes impossible when d grows.

Representation in code. In numerical computing, a tensor is typically stored as an `ndarray` (NumPy array).

Definition 2.2 (Mode- k unfolding). Let $A \in \mathbb{R}^{n_1 \times \dots \times n_d}$.

The k -th unfolding (or matricization) of A is the matrix

$$A^{(k)} \in \mathbb{R}^{(n_1 \dots n_k) \times (n_{k+1} \dots n_d)}$$

obtained by reshaping the tensor so that the first k indices form the row index and the remaining indices form the column index.

Explanation (informal). Unfolding allows us to convert a tensor with arbitrary dimensions into a single matrix by reshaping (similarly to how a matrix can be represented by a regular array given proper indexing). This is important because matrices can be approximated by low-rank factorizations using the Singular Value Decomposition (SVD).

The SVD decomposition is of the form

$$A = U\Sigma V^\top,$$

and truncating small singular values yields the best low-rank approximation in the Frobenius norm.

TT decomposition is built from successive low-rank approximations of unfolding matrices [Ose11].

Representation in code. In NumPy, unfolding is just a reshape:

```
A_k = A.reshape(
    np.prod(A.shape[:k]),
    np.prod(A.shape[k:])
)
```

3 Tensor Train Decomposition

3.1 Definition

Definition 3.1 (Tensor Train decomposition (TTD)). A tensor $A \in \mathbb{R}^{n_1 \times \dots \times n_d}$ is in Tensor Train (TT) format if it can be written as

$$A(i_1, \dots, i_d) = G_1(i_1)G_2(i_2) \cdots G_d(i_d),$$

where for each k :

$$G_k(i_k) \in \mathbb{R}^{r_{k-1} \times r_k},$$

and

$$r_0 = r_d = 1.$$

Equivalently, treating matrices as a function $\mathbb{R}^2 \rightarrow \mathbb{R}$:

$$A(i_1, \dots, i_d) = \sum_{\alpha_1, \dots, \alpha_{d-1}} G_1(i_1)(1, \alpha_1)G_2(i_2)(\alpha_1, \alpha_2) \cdots G_d(i_d)(\alpha_{d-1}, 1).$$

Remark 3.1. The storage complexity of the TT format is

$$\sum_{k=1}^d n_k r_{k-1} r_k.$$

If $r_k \leq r$ and $n_k = n$, this becomes

$$\mathcal{O}(dnr^2),$$

which is linear in d . Thus, TT format breaks the curse of dimensionality when TT-ranks remain moderate.

Explanation (informal). Each index i_k selects a matrix $G_k(i_k)$. Multiplying these matrices yields a scalar because the boundary ranks satisfy $r_0 = r_d = 1$.

Thus, the tensor entry is obtained by a chain of matrix multiplications.

Explanation (informal). Instead of storing the full tensor, we store small 3D tensors called **cores**. Each core connects only to its neighbors, forming a chain (a “train”). Instead of storing all n^d elements, only approximately $\mathcal{O}(dnr^2)$ numbers are required when ranks r_k are moderate [Ose11].

Example (TT representation). Let $A \in \mathbb{R}^{2 \times 2 \times 2}$ be defined by

$$A(i_1, i_2, i_3) = i_1 + i_2 + i_3.$$

A TT representation writes

$$A(i_1, i_2, i_3) = G_1(i_1) G_2(i_2) G_3(i_3),$$

with

$$G_1(i_1) \in \mathbb{R}^{1 \times r_1}, \quad G_2(i_2) \in \mathbb{R}^{r_1 \times r_2}, \quad G_3(i_3) \in \mathbb{R}^{r_2 \times 1}.$$

Explanation (informal). The tensor is not stored entrywise. Instead, each index selects a small matrix, and multiplying these matrices produces the scalar value.

Explanation (informal). For structured tensors (such as sums of variables), small ranks r_k are sufficient. This yields a compressed representation.

Representation in code. In practice, TT format is stored as a list of cores:

```
# TT tensor represented as list of cores
G = [G1, G2, ..., Gd]
# Each core has shape (r_{k-1}, n_k, r_k)
Gk.shape = (r_prev, n_k, r_next)
```

3.2 TT-rank and related concepts

Definition 3.2 (TT-rank). Let $A^{(k)}$ denote the k -th unfolding of A . The TT-ranks are defined as

$$r_k = \text{rank}\left(A^{(k)}\right), \quad k = 1, \dots, d-1.$$

Explanation (informal). The rank r_k measures how strongly the first k variables interact with the remaining $d - k$ variables. If $r_k = 1$, the first k variables are independent of the rest.

These ranks determine the minimal possible internal dimensions of any TT representation.

4 Basic Operations in TT Format

4.1 Computing TTD

4.1.1 From Unfoldings to TT-SVD

Definition 4.1 (Frobenius norm of tensor). For a tensor A ,

$$\|A\|_F = \sqrt{\sum_{i_1, \dots, i_d} A(i_1, \dots, i_d)^2}.$$

Conceptual idea. The TT-SVD algorithm constructs a TT representation by applying truncated SVD to successive unfolding matrices.

At each step:

1. Reshape the tensor into a matrix.
2. Compute its truncated SVD.
3. Store left singular vectors as a TT core.
4. Pass the remainder to the next step.

This produces a quasi-optimal low-rank TT approximation.

4.1.2 Algorithm for computing a TT decomposition (TT-SVD)

The TT-SVD algorithm constructs matrix-valued functions

$$G_k : \{1, \dots, n_k\} \rightarrow \mathbb{R}^{r_{k-1} \times r_k}$$

by sequential low-rank approximation of unfolding matrices [Ose11] as described in algorithm 1.

Algorithm 1 TT-SVD in matrix-valued function notation

Input: Tensor $A \in \mathbb{R}^{n_1 \times \dots \times n_d}$, tolerance ε

Output: Matrix-valued functions $G_1, \dots, G_d$

- 1: Set $r_0 \leftarrow 1$
- 2: Set $C \leftarrow A$
- 3: Set $\delta \leftarrow \frac{\varepsilon}{\sqrt{d-1}} \|A\|_F$
- 4: **for** $k = 1$ to $d - 1$ **do**
- 5: Reshape C into matrix

$$C \in \mathbb{R}^{(r_{k-1}n_k) \times (n_{k+1} \dots n_d)}$$

- 6: Compute truncated SVD

$$C \approx U \Sigma V^\top, \quad \|C - U \Sigma V^\top\|_F \leq \delta$$

- 7: Let r_k be the number of kept singular values
- 8: Partition U into n_k consecutive blocks of size $r_{k-1} \times r_k$
- 9: Define matrix-valued function

$$G_k(i_k) := \text{the block of } U \text{ corresponding to index } i_k$$

- 10: Set $C \leftarrow \Sigma V^\top$
- 11: **end for**
- 12: Define $G_d(i_d)$ from the final matrix C

$$C \in \mathbb{R}^{r_{d-1} \times n_d}$$

by taking column i_d as a matrix in $\mathbb{R}^{r_{d-1} \times 1}$.

Remark 4.1. A standard relative-accuracy choice is to distribute the Frobenius error budget across the $d - 1$ SVD truncations:

$$\delta = \frac{\varepsilon}{\sqrt{d-1}} \|A\|_F,$$

so that the final TT approximation satisfies $\|A - \hat{A}\|_F \leq \varepsilon \|A\|_F$ (see the TT-SVD error bound in [Ose11]).

Explanation (informal). At step k we reshape the tensor into a matrix that separates the first k indices from the remaining $d - k$ indices. This matrix describes how these two groups of indices depend on each other.

We then compute a truncated Singular Value Decomposition (SVD) of this matrix to obtain a low-rank approximation. The rank r_k of this approximation determines the next TT-rank.

The left singular vectors are reshaped into blocks that form the k -th TT core $G_k(i_k)$. The remaining part of the tensor is passed to the next step, where the same procedure is applied again.

By repeating this process for $k = 1, \dots, d - 1$, the algorithm constructs all TT cores and determines the TT-ranks of the representation.

In this way, TT-SVD builds the tensor train representation sequentially, extracting one core at a time.

Example (first TT-SVD step). Let $A \in \mathbb{R}^{2 \times 3 \times 2}$.

At step $k = 1$, reshape:

$$A^{(1)} \in \mathbb{R}^{2 \times 6}.$$

Compute truncated SVD:

$$A^{(1)} \approx U \Sigma V^\top.$$

Let r_1 be the number of retained singular values.

Define:

$$G_1(i_1) = \text{block of } U \text{ corresponding to } i_1,$$

and set

$$C = \Sigma V^\top.$$

Explanation (informal). We separate the first variable from the remaining ones and approximate their interaction by a low-rank matrix.

Explanation (informal). The matrix U encodes how i_1 interacts with the rest. The remainder C is passed to the next step.

Code slot (NumPy).

TODO implemented TTD

4.2 Rounding

4.2.1 Algorithm for rounding (rank truncation / recompression)

Rounding (also called recompression) reduces inflated TT-ranks after operations (addition, etc.) while controlling the approximation error. The standard procedure is to orthogonalize (QR sweep) and then truncate (SVD sweep) [Ose11] as shown in algorithm 2.

Remark 4.2. The QR sweep makes the partial products

$$G_1(i_1) \cdots G_k(i_k)$$

orthonormal in Frobenius norm.

This ensures that the subsequent SVD truncations behave like optimal matrix approximations, leading to numerical stability and quasi-optimal error control.

Algorithm 2 TT rounding in matrix-valued function notation

Input: Matrix-valued functions $G_1, \dots, G_d$, tolerance ε

Output: Rounded matrix-valued functions $\tilde{G}_1, \dots, \tilde{G}_d$

1: Set $\delta \leftarrow \frac{\varepsilon}{\sqrt{d-1}} \|\hat{A}\|_F$

2: **Left-to-right orthogonalization**

3: **for** $k = 1$ to $d - 1$ **do**

4: Form matrix

$$M_k := \begin{bmatrix} G_k(1) \\ \vdots \\ G_k(n_k) \end{bmatrix} \in \mathbb{R}^{(r_{k-1}n_k) \times r_k}$$

5: Compute thin QR:

$$M_k = Q_k R_k$$

6: Redefine $G_k(i_k)$ as corresponding block of Q_k

7: Update next core:

$$G_{k+1}(i_{k+1}) \leftarrow R_k G_{k+1}(i_{k+1})$$

8: **end for**

9: **Right-to-left truncation**

10: **for** $k = d$ down to 2 **do**

11: Form matrix

$$N_k := \begin{bmatrix} G_k(1) & \cdots & G_k(n_k) \end{bmatrix} \in \mathbb{R}^{r_{k-1} \times (n_k r_k)}$$

12: Compute truncated SVD

$$N_k \approx U \Sigma V^\top$$

13: Update:

$$G_k(i_k) \leftarrow \text{corresponding block of } V^\top$$

14: Update previous core:

$$G_{k-1}(i_{k-1}) \leftarrow G_{k-1}(i_{k-1})(U \Sigma)$$

15: **end for**

Important distinction.

- TT-SVD constructs a TT representation from a full tensor.
- Rounding reduces the ranks of an existing TT tensor while controlling the approximation error.

Example (rank growth and truncation). Let A and B be TT tensors with ranks

$$r^A = (1, 3, 3, 1), \quad r^B = (1, 4, 4, 1).$$

After addition:

$$r^C = (1, 7, 7, 1).$$

After rounding:

$$r^C \rightarrow (1, 3, 3, 1).$$

Explanation (informal). Basic operations increase TT-ranks. This leads to higher storage and computational cost.

Explanation (informal). Rounding reduces ranks while controlling the approximation error. It restores a compact representation.

Note. The Frobenius norm satisfies $\|A\|_F = \sqrt{\langle A, A \rangle}$, so it reduces to an inner product computation (see section 4.4).

Code slot (NumPy).

TODO implemented rounding

4.3 Addition

Given A and B in TT format, a standard construction forms a TT representation of $C = A + B$ by stacking cores blockwise; ranks add (then we round).

Construction (conceptual). If A has cores $G_A^{(k)} \in \mathbb{R}^{r_{k-1}^A \times n_k \times r_k^A}$ and B has $G_B^{(k)} \in \mathbb{R}^{r_{k-1}^B \times n_k \times r_k^B}$, define cores of C :

- For $k = 1$: concatenate along the last (right-rank) index:

$$G_C^{(1)}(1, i_1, :) = [G_A^{(1)}(1, i_1, :) \quad G_B^{(1)}(1, i_1, :)]$$

- For $1 < k < d$: block-diagonal stacking:

$$G_C^{(k)}(:, i_k, :) = \begin{bmatrix} G_A^{(k)}(:, i_k, :) & 0 \\ 0 & G_B^{(k)}(:, i_k, :) \end{bmatrix}$$

- For $k = d$: concatenate along the first (left-rank) index:

$$G_C^{(d)}(:, i_d, 1) = \begin{bmatrix} G_A^{(d)}(:, i_d, 1) \\ G_B^{(d)}(:, i_d, 1) \end{bmatrix}$$

This yields ranks $r_k^C = r_k^A + r_k^B$. The procedure is finalized by rounding C (algorithm 2).

Code slot.

then call tt_round(...)

Algorithm 3 Inner product in TT format

Input: Matrix-valued functions $G_1, \dots, G_d$ **Output:** Inner product $\langle A, B \rangle$

- 1: Set $M_0 := [1]$ $\triangleright 1 \times 1$ matrix
 - 2: **for** $k = 1$ **to** d **do**
 - 3: $M_k = \sum_{i_k} G_k(i_k)^T M_{k-1} H_k(i_k)$ $\triangleright M_k$ is $r_k \times r_k$
 - 4: **end for**
 - 5: **return** M_d $\triangleright M_d$ is a 1×1 matrix
-

Scalar multiplication. For $c \in \mathbb{R}$ and TT tensor A with cores $G^{(k)}$, one may scale a single core:

$$(cA) \text{ can be represented by cores } (cG^{(1)}, G^{(2)}, \dots, G^{(d)}).$$

Code slot.

```
# - scalar multiply: scale G[0] *= c
```

Code slot.

```
# TODO implemented proper norm_F
```

4.4 Inner products

Definition 4.2 (Inner product in TT format). Let A and B be two tensors in TT format with matrix-valued cores

$$G_k : \{1, \dots, n_k\} \rightarrow \mathbb{R}^{r_{k-1} \times r_k}, \quad H_k : \{1, \dots, n_k\} \rightarrow \mathbb{R}^{s_{k-1} \times s_k}.$$

The Frobenius inner product

$$\langle A, B \rangle = \sum_{i_1, \dots, i_d} A(i_1, \dots, i_d) B(i_1, \dots, i_d)$$

can be computed sequentially by summing the inner products of the cores contracted by matrices M_k as shown in algorithm 3.

Define $M_0 = [1]$ and recursively compute

$$M_k = \sum_{i_k=1}^{n_k} G_k(i_k)^\top M_{k-1} H_k(i_k), \quad k = 1, \dots, d.$$

Then

$$\langle A, B \rangle = M_d.$$

Explanation (informal). Instead of summing over all $n_1 \cdots n_d$ tensor entries, we contract the tensor cores one dimension at a time.

At each step k , we accumulate a small matrix M_k that carries the interaction between the first k dimensions. Since the boundary ranks satisfy $r_0 = s_0 = 1$, the final matrix M_d has size 1×1 and thus can be interpreted as a scalar. This reduces exponential complexity to polynomial complexity.

Complexity. If $n_k \leq n$ and all TT-ranks are bounded by r , the computational complexity reduces from exponential $\mathcal{O}(n^d)$ to polynomial $\mathcal{O}(dnr^3)$.

Code slot.

inner product

4.5 Matrix-vector multiplication

Definition 4.3 (TT matrix applied to TT vector). A linear operator in TT format is represented by matrix-valued functions

$$A_k : \{1, \dots, n_k\} \times \{1, \dots, m_k\} \rightarrow \mathbb{R}^{R_{k-1} \times R_k}.$$

A TT vector is represented by cores

$$G_k : \{1, \dots, m_k\} \rightarrow \mathbb{R}^{r_{k-1} \times r_k}.$$

The product $y = Ax$ is defined by

$$y(i_1, \dots, i_d) = \sum_{j_1, \dots, j_d} A(i_1, j_1, \dots, i_d, j_d) x(j_1, \dots, j_d).$$

The resulting TT cores are

$$\tilde{G}_k(i_k) = \sum_{j_k=1}^{m_k} A_k(i_k, j_k) \otimes G_k(j_k),$$

where $\otimes$ denotes the Kronecker product.

Explanation (informal). Each core of the operator interacts only with the corresponding core of the vector at the same dimension.

For each dimension k we combine:

- the operator slice $A_k(i_k, j_k)$
- the vector slice $G_k(j_k)$

This produces a new core whose ranks are multiplied:

$$\tilde{r}_k \approx R_k r_k.$$

Afterward, TT rounding is typically applied to control rank growth.

Code slot.

```
# - represent TT-matrix cores Acores[k] with shape (R_{k-1}, n_k, m_k, R_k)
# - represent TT-vector cores X[k] with shape (r_{k-1}, m_k, r_k)
# - build Y[k] by contracting over m_k, ranks multiply
# - round Y
```

4.6 Flipping an axis

Definition (axis flip). Let $A \in \mathbb{R}^{n_1 \times \dots \times n_d}$ be in TT format with cores

$$G_k \in \mathbb{R}^{r_{k-1} \times n_k \times r_k}.$$

Flipping axis k produces a tensor $\tilde{A}$ such that

$$\tilde{A}(i_1, \dots, i_k, \dots, i_d) = A(i_1, \dots, n_k - i_k + 1, \dots, i_d).$$

Explanation (informal). Flipping an axis corresponds to reversing the ordering of indices along that dimension.

Since each dimension is represented independently in TT format, the operation affects only a single core.

Construction. The flipped tensor is obtained by reversing the index dimension of the k -th core:

$$\tilde{G}_k(:, i_k, :) = G_k(:, n_k - i_k + 1, :).$$

All other cores remain unchanged:

$$\tilde{G}_j = G_j, \quad j \neq k.$$

Remark. The TT-ranks are unchanged by this operation.

Complexity. The operation requires $\mathcal{O}(r_{k-1}n_kr_k)$ operations.

Code slot (NumPy).

#TODO implement

4.7 Element access

Definition 4.4 (Element evaluation). Let A be a TT tensor with matrix-valued cores

$$G_k : \{1, \dots, n_k\} \rightarrow \mathbb{R}^{r_{k-1} \times r_k}.$$

The entry

$$A(i_1, \dots, i_d)$$

can be computed by sequential multiplication:

$$v_0 = 1, \quad v_k = v_{k-1}G_k(i_k), \quad k = 1, \dots, d.$$

Then

$$A(i_1, \dots, i_d) = v_d.$$

Explanation (informal). Instead of reconstructing the full tensor, we select one slice from each core and multiply them from left to right.

Each step propagates a small matrix (or vector), and since $r_0 = r_d = 1$, the final result is a scalar.

Complexity. If TT-ranks are bounded by r , the cost is

$$\mathcal{O}(dr^2).$$

Code slot.

TODO: element access

4.8 Mode permutation (transpose)

Definition 4.5 (Mode permutation in TT format). Let $A \in \mathbb{R}^{n_1 \times \dots \times n_d}$ be represented in TT format with cores $G_1, \dots, G_d$.

Given a permutation

$$\sigma : \{1, \dots, d\} \rightarrow \{1, \dots, d\},$$

the permuted tensor $\tilde{A}$ is defined by

$$\tilde{A}(i_1, \dots, i_d) = A(i_{\sigma(1)}, \dots, i_{\sigma(d)}).$$

A TT representation of $\tilde{A}$ is obtained by reordering and recompressing the TT cores such that the tensor remains in TT format.

Explanation (informal). Unlike basic operations such as scaling or axis flipping, a general permutation cannot be performed by simply reordering the TT cores.

This is because each core encodes dependencies between neighboring dimensions.

Changing the order of dimensions changes these dependencies, so the TT decomposition must be recomputed locally.

Construction (adjacent swap). A general permutation is implemented as a sequence of swaps of neighboring dimensions.

To swap dimensions k and $k + 1$:

1. Contract the two neighboring cores:

$$T(i_k, i_{k+1}) = G_k(i_k) G_{k+1}(i_{k+1})$$

2. Reshape T into a matrix:

$$T \in \mathbb{R}^{(r_{k-1} n_{k+1}) \times (n_k r_{k+1})}$$

3. Compute a truncated SVD:

$$T \approx U \Sigma V^\top$$

4. Reshape U and $V^\top$ into new TT cores:

$$\tilde{G}_k(i_{k+1}), \quad \tilde{G}_{k+1}(i_k)$$

Remark. Mode permutation generally increases TT-ranks before truncation. Therefore, rounding (algorithm 2) is required to maintain a compact representation.

Complexity. A single adjacent swap has complexity

$$\mathcal{O}(n_k n_{k+1} r^3).$$

A general permutation requires $\mathcal{O}(d^2)$ such swaps.

Code slot.

```
# TODO: implement adjacent swap of TT cores
# then compose swaps to achieve general permutation
```

4.9 Tensor contraction (tensordot)

Definition 4.6 (Tensor contraction in TT format). Let A and B be TT tensors.

A tensor contraction (tensordot) along selected modes sums over shared indices of A and B .

In the special case where all modes are contracted, this reduces to the Frobenius inner product:

$$\langle A, B \rangle = \sum_{i_1, \dots, i_d} A(i_1, \dots, i_d) B(i_1, \dots, i_d).$$

Remark (ordering of contracted modes). Efficient contraction in TT format requires the contracted modes to form a contiguous block of TT cores.

More precisely, the contracted cores should appear consecutively and include a boundary core:

- either starting from the first core,
- or ending at the last core.

If the required contraction does not satisfy this ordering, the TT cores must first be permuted (see section 4.8) before performing the contraction.

Explanation (informal). The contraction is performed sequentially across dimensions.

At each dimension k , we combine slices of A and B , sum over the contracted index, and propagate a small matrix that carries the accumulated interaction between all previously contracted dimensions.

More precisely, if M_{k-1} denotes the propagated matrix from the previous step, the next contraction has the form

$$M_k = \sum_{i_k} G_k(i_k)^\top M_{k-1} H_k(i_k),$$

which reduces the contribution of dimension k into a new small matrix M_k .

This avoids forming the full tensors explicitly.

Important note. General contractions increase TT-ranks. This is mostly due to the fact that we need to permute the cores before performing the tensordot. Therefore, rounding (algorithm 2) is applied afterward.

Complexity. If $n_k \leq n$ and ranks are bounded by r , the cost is

$$\mathcal{O}(dnr^3).$$

Code slot.

TODO: tensordot / contraction

5 TT-Matrix Format

5.1 TT-Matrix representation

Definition 5.1 (TT-matrix). A linear operator

$$A \in \mathbb{R}^{(n_1 \cdots n_d) \times (m_1 \cdots m_d)}$$

is said to be in TT-matrix format if it is represented by matrix-valued functions

$$A_k : \{1, \dots, n_k\} \times \{1, \dots, m_k\} \rightarrow \mathbb{R}^{R_{k-1} \times R_k},$$

such that

$$A(i_1, \dots, i_d, j_1, \dots, j_d) = A_1(i_1, j_1) \cdots A_d(i_d, j_d),$$

with $R_0 = R_d = 1$.

Explanation (informal). A TT-matrix represents a high-dimensional linear operator as a sequence of small core tensors.

Each core connects input indices j_k and output indices i_k at dimension k , while also propagating low-rank coupling information via auxiliary indices.

This representation reduces storage from exponential $\mathcal{O}(n^d m^d)$ to polynomial $\mathcal{O}(dnmR^2)$.

5.2 Differential operators in TT format

5.2.1 Gradient operator

Definition 5.2 (Gradient in TT format). Let $A \in \mathbb{R}^{n_1 \times \cdots \times n_d}$ be a TT tensor.

For each $k = 1, \dots, d$, define a TT-matrix operator

$$D^{(k)} = I \otimes \cdots \otimes D_k \otimes \cdots \otimes I,$$

where $D_k \in \mathbb{R}^{n_k \times n_k}$ is a 1D differentiation matrix.

The partial derivative is defined by

$$\frac{\partial A}{\partial x_k} = D^{(k)} A,$$

where the product is computed using TT matrix-vector multiplication.

The gradient is given by

$$\nabla A = \left(D^{(1)} A, \dots, D^{(d)} A \right).$$

Explanation (informal). Each derivative acts only along one dimension.

The TT-matrix $D^{(k)}$ inserts the differentiation matrix D_k into the k -th position of a Kronecker product.

Applying the operator corresponds to TT matrix-vector multiplication, which preserves the tensor structure.

Important note. Although the operator $D^{(k)}$ has low TT-rank, the resulting tensor $D^{(k)} A$ typically has increased TT-ranks.

Therefore, rounding (algorithm 2) is required.

5.2.2 Laplace operator

Definition 5.3 (Laplace operator in TT format). The Laplace operator is defined as

$$\Delta = \sum_{k=1}^d \frac{\partial^2}{\partial x_k^2}.$$

Each term is represented as a TT-matrix

$$L^{(k)} = I \otimes \dots \otimes D_k^{(2)} \otimes \dots \otimes I,$$

where $D_k^{(2)}$ is a second-order differentiation matrix.

Thus,

$$\Delta = \sum_{k=1}^d L^{(k)}.$$

Explanation (informal). The Laplacian is a sum of independent second derivatives.

Each term has a simple tensor-product structure, which admits a low-rank TT representation.

The sum increases TT-ranks linearly in d , and rounding is applied after operator evaluation.

5.3 TT-matrix applied to TT tensor

Definition 5.4 (TT-matrix times TT tensor). Let a linear operator A be given in TT-matrix format with cores

$$A_k : \{1, \dots, n_k\} \times \{1, \dots, m_k\} \rightarrow \mathbb{R}^{R_{k-1} \times R_k},$$

and let a tensor X be given in TT format with cores

$$G_k : \{1, \dots, m_k\} \rightarrow \mathbb{R}^{r_{k-1} \times r_k}.$$

The product $Y = AX$ is defined componentwise by

$$Y(i_1, \dots, i_d) = \sum_{j_1, \dots, j_d} A(i_1, j_1, \dots, i_d, j_d) X(j_1, \dots, j_d).$$

The resulting tensor Y admits a TT representation with cores

$$\tilde{G}_k(i_k) = \sum_{j_k=1}^{m_k} A_k(i_k, j_k) \otimes G_k(j_k),$$

where $\otimes$ denotes the Kronecker product.

Algorithm 4 TT-matrix applied to TT tensor

Input: TT-matrix cores $A_1, \dots, A_d$, TT cores $G_1, \dots, G_d$

Output: TT cores $\tilde{G}_1, \dots, \tilde{G}_d$

- 1: **for** $k = 1$ to d **do**
- 2: **for** $i_k = 1$ to n_k **do**
- 3: Compute

$$\tilde{G}_k(i_k) = \sum_{j_k=1}^{m_k} A_k(i_k, j_k) \otimes G_k(j_k)$$

- 4: **end for**
 - 5: **end for**
 - 6: Apply TT rounding (algorithm 2) to $\tilde{G}_1, \dots, \tilde{G}_d$
-

Explanation (informal). The operator acts independently on each dimension.

At dimension k , we combine:

- the operator slice $A_k(i_k, j_k)$,
- the tensor slice $G_k(j_k)$,

and sum over the index j_k .

The Kronecker product couples the operator rank R_k with the tensor rank r_k , producing new ranks

$$\tilde{r}_k \approx R_k r_k.$$

Important note. The TT-ranks grow multiplicatively during this operation.

Therefore, rounding (algorithm 2) is required to obtain a compact representation and control the error.

Complexity. If $n_k \leq n$, $m_k \leq n$, and all ranks are bounded by r and R , then the computational cost is

$$\mathcal{O}(dnR^2r^2).$$

Code slot.

```
# TODO: TT-matrix x TT-tensor
# - loop over cores
# - contract over j_k
# - use kron(A_k slice, G_k slice)
# - round result
```

6 PDEs and Using TT format to solve them

6.1 Definition of PDE

Definition 6.1 (Partial Differential Equation). Let $\Omega \subset \mathbb{R}^d$. A partial differential equation (PDE) has the form

$$\mathcal{L}u(x_1, \dots, x_d) = f(x_1, \dots, x_d),$$

where

- $u : \Omega \rightarrow \mathbb{R}$ is the unknown function,
- $\mathcal{L}$ is a differential operator,
- f is a given function.

Example (Poisson equation):

$$-\Delta u = f, \quad \Delta = \sum_{k=1}^d \frac{\partial^2}{\partial x_k^2}.$$

Explanation (informal). A PDE describes how a function of several variables changes in space.

For example, the Poisson equation models heat distribution, electrostatics, or diffusion.

In low dimensions ($d = 1, 2, 3$), standard numerical methods work well. However, when d becomes large (for example $d = 6, 8, 10$), the computational cost grows rapidly.

Tensor Train methods allow us to represent both:

- the solution u ,
- the operator $\mathcal{L}$,

in compressed TT form.

6.2 Discretization and the Curse of Dimensionality

Definition 6.2 (Finite-dimensional discretization). Discretize each variable x_k using n grid points.

The continuous function $u(x_1, \dots, x_d)$ is replaced by a tensor

$$U \in \mathbb{R}^{n \times \dots \times n} = \mathbb{R}^{n^d}.$$

The differential operator $\mathcal{L}$ is replaced by a matrix

$$A \in \mathbb{R}^{n^d \times n^d}.$$

The PDE becomes the linear system

$$AU = F.$$

Explanation (informal). After discretization, solving a PDE means solving a linear system. If each dimension has n grid points, the number of unknowns is n^d .

Even for moderate values, for example $n = 50$ and $d = 8$, the system has $50^8 \approx 4 \times 10^{13}$ unknowns — far beyond what can be stored. This exponential growth is the result of the curse of dimensionality.

6.3 PDEs with Constant Coefficients

Definition 6.3 (Constant-coefficient PDE). A linear second-order elliptic PDE has constant coefficients if it can be written in the form

$$-\sum_{k=1}^d c_k \frac{\partial^2 u}{\partial x_k^2} = f(x),$$

where the coefficients $c_k \in \mathbb{R}$ are constants (independent of x).

In the special case $c_k = 1$ for all k , this reduces to the Poisson equation

$$-\Delta u = f.$$

Discretized form. After discretization on a tensor grid with n points per dimension, the PDE becomes a linear system

$$AU = F,$$

where

$$A = \sum_{k=1}^d I \otimes \cdots \otimes L \otimes \cdots \otimes I,$$

and $L \in \mathbb{R}^{n \times n}$ is the one-dimensional discrete Laplacian.

Explanation (informal). When coefficients are constant, each spatial direction contributes the same type of operator.

This leads to a separable Kronecker structure in the discretized matrix A .

As a result:

- A has low TT-rank,
- it can be stored efficiently in TT format,
- high-dimensional problems become tractable.

The structure of the operator does not depend on position, which is what enables this strong separability.

6.4 PDEs with Variable Coefficients

Definition 6.4 (Variable-coefficient PDE). A second-order elliptic PDE has variable coefficients if it has the form

$$-\nabla \cdot (a(x) \nabla u(x)) = f(x),$$

where the coefficient function

$$a : \Omega \rightarrow \mathbb{R}$$

depends on the spatial variables.

Discretized form. After discretization, we again obtain a linear system

$$A(a)U = F,$$

but the matrix $A(a)$ now depends on the spatially varying coefficient.

Explanation (informal). When $a(x)$ varies in space, the operator is no longer a simple Kronecker sum.

However, if the coefficient function admits a low-rank tensor approximation, for example

$$a(x_1, \dots, x_d) \approx \sum_{\alpha} a_1(x_1) \cdots a_d(x_d),$$

then the discretized operator $A(a)$ can still be approximated in TT format.

Thus, TT methods extend beyond constant-coefficient problems.

6.5 Structural Lattice of PDEs

Definition 6.5 (Structured PDE components). A general linear PDE can be written as

$$\mathcal{L}u = -\nabla \cdot (K(x) \nabla u) + b(x) \cdot \nabla u + c(x)u = f(x),$$

where

- $K(x) \in \mathbb{R}^{d \times d}$ is the diffusion matrix,
- $b(x) \in \mathbb{R}^d$ is the advection (transport) field,
- $c(x)$ is a reaction term,

- $f(x)$ is the inhomogeneity.

Each component may be:

- constant or variable,
- scalar or vector/matrix-valued,
- separable or non-separable.

We can organize PDEs by increasing structural complexity:

constant coefficients $\subset$ separable variable coefficients $\subset$ fully variable coefficients.

Explanation (informal). A PDE is not just “one object” — it is composed of several building blocks: diffusion, transport, reaction, and forcing.

Each of these components can vary in complexity. This naturally induces a *hierarchy (lattice)* of PDE classes.

7 Implicit Methods for Solving PDE

7.1 Implicit Method

Definition 7.1 (Implicit time discretization). Consider a time-dependent PDE of the form

$$\frac{\partial u}{\partial t} = \mathcal{L}u + f, \quad u(0) = u_0.$$

Let $t_n = n\Delta t$.

An implicit time-stepping scheme computes u^{n+1} by solving an equation involving the unknown future value:

$$\Phi(u^{n+1}, u^n) = 0.$$

In general form:

$$u^{n+1} = u^n + \Delta t \mathcal{L}(u^{n+1}) + \Delta t f^{n+1}.$$

Explanation (informal). In an implicit method, the new solution u^{n+1} appears on both sides of the equation. This means we must solve a system of equations at every time step.

Advantages:

- Stable for large time steps
- Often unconditionally stable
- Suitable for stiff PDEs

Disadvantages:

- Requires solving linear systems
- Computationally more expensive per step

7.2 Method A

8 Explicit Methods for Solving PDE

8.1 Explicit Method

Definition 8.1 (Explicit time discretization). An explicit time-stepping scheme computes u^{n+1} using only already known quantities:

$$u^{n+1} = u^n + \Delta t \mathcal{L}(u^n) + \Delta t f^n.$$

Explanation (informal). In an explicit method, the new value u^{n+1} is computed directly from previously known values.

Advantages:

- No linear system to solve
- Cheap per time step
- Easy to implement

Disadvantages:

- Stability restrictions on Δt
- CFL condition
- Often requires very small time steps

8.2 Method A

9 Conclusion and Further Reading

References

- [Ose09] Ivan V. Oseledets. Breaking the curse of dimensionality. *SIAM Journal on Scientific Computing*, 2009.
- [Ose11] Ivan V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 2011.